

Desenvolvimento para Dispositivos Móveis

Armazenamento local

Prof. Chauã Queirolo | <https://github.com/chaue>

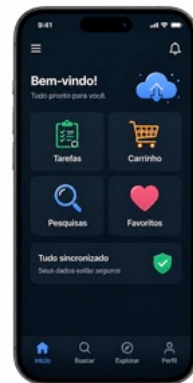
1

Sumário

2

Introdução

- Um app perde todos os dados quando é fechado?
- Exemplos:
 - Tema escuro ativado
 - Usuário logado
 - Lista de tarefas offline
 - Produtos no carrinho
 - Histórico de pesquisas
 - Favoritos



**Sem persistência, tudo isso
desaparece ao reiniciar o
app**

O que é armazenamento local?

- É o armazenamento feito no próprio dispositivo do usuário.
- O app grava dados localmente para usar depois.
- Configurações
- Cache
- Rascunhos
- Dados offline
- Preferências do usuário
- Sessão local

5

Tipos de armazenamento local

Tipo	Exemplo	Uso
Chave-valor simples	SharedPreferences	Configurações pequenas
Banco relacional	SQLite / Floor	Dados estruturados
NoSQL local	Hive	Objetos, cache, dados flexíveis

6

Quando usar cada opção?

- SharedPreferences:
- Tema escuro
- Idioma escolhido
- Primeiro acesso ao app
- Pequenas flags

Quando usar cada opção?

- Floor / SQLite:
- Clientes
- Pedidos
- Produtos
- Tarefas com filtros
- Dados com relações

Quando usar cada opção?

- Hive:
- Cache local
- Objetos simples
- Favoritos
- Dados offline rápidos
- Estruturas menos rígidas

Regra prática

Se for uma configuração simples

Use SharedPreferences

Se precisar de tabela, filtro, ordenação e relacionamento

Use SQLite com Floor

Se precisar salvar objetos localmente de forma rápida e flexível

Use Hive

SharedPreferences

- SharedPreferences armazena dados em pares chave-valor
- Use para dados pequenos e simples.
- Não use para dados sensíveis ou críticos.

```
"darkMode" = true
```

```
"language" = "pt_BR"
```

```
"firstAccess" = false
```

11

Instalando SharedPreferences

No terminal:

```
flutter pub add shared_preferences
```

Importação:

```
import 'package:shared_preferences/shared_preferences.dart';
```

12

SharedPreferencesAsync

```
final prefs = SharedPreferencesAsync();

await prefs.setBool('darkMode', true);

final bool darkMode =
    await prefs.getBool('darkMode') ?? false;
```

13

Serviço de preferências

```
import 'package:shared_preferences/shared_preferences.dart';

class SettingsService {
  final SharedPreferencesAsync _prefs = SharedPreferencesAsync();

  static const String _darkModeKey = 'darkMode';

  Future<void> saveDarkMode(bool value) async {
    await _prefs.setBool(_darkModeKey, value);
  }

  Future<bool> getDarkMode() async {
    return await _prefs.getBool(_darkModeKey) ?? false;
  }
}
```

14

Usando no Flutter

```
@override
void initState() {
  super.initState();
  loadSettings();
}

Future<void> loadSettings() async {
  final value = await service.getDarkMode();
  setState(() {
    darkMode = value;
  });
}

Future<void> changeDarkMode(bool value) async {
  await service.saveDarkMode(value);
  setState(() {
    darkMode = value;
  });
}
```

15

Limitações do SharedPreferences

- **Não é ideal para:**
 - Listas grandes
 - Dados relacionais
 - Dados sensíveis
 - Consultas complexas
 - Histórico de muitos registros
 - Dados críticos de negócio
- **Exemplo ruim:**
 - Salvar todos os pedidos de um sistema em SharedPreferences.

16

SQLite

- SQLite é um banco relacional local.
- Ele armazena dados em tabelas.
- :

Tabela: tasks

id	title	done
1	Estudar Flutter	false
2	Fazer exercício	true

Por que usar SQLite?

- SQLite é útil quando os dados têm estrutura clara.
- Exemplos:
 - Uma tarefa tem id, título e status
 - Um pedido tem itens
 - Um cliente tem endereço
 - Um produto tem preço e categoria

Floor

- Floor é uma abstração sobre SQLite para Flutter.
- Ele reduz código repetitivo e usa anotações.
- Componentes principais:
 - Entity
 - DAO
 - Database
 - Generated code
 - Queries SQL

Instalando Floor

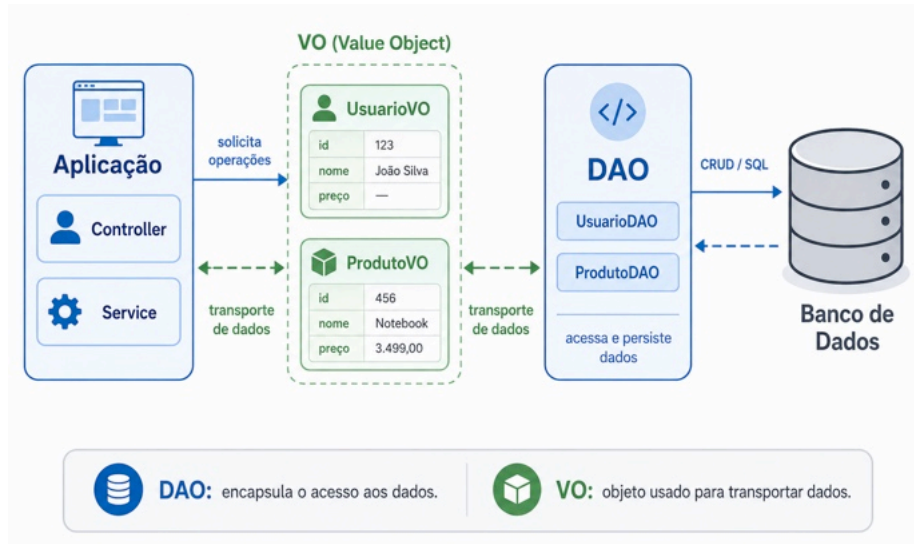
No pubspec.yaml:

```
dependencies:  
  floor: ^0.19.1  
  sqflite: ^2.4.2  
  path: ^1.9.0  
  
dev_dependencies:  
  floor_generator: ^0.19.1  
  build_runner: ^2.4.15
```

Depois:

```
flutter pub get
```

Padrão DAO/VO



21

Entity no Floor

```
import 'package:floor/floor.dart';

@Entity
class TaskEntity {
  @primaryKey
  final int? id;

  final String title;
  final bool done;

  TaskEntity({
    this.id,
    required this.title,
    required this.done,
  });
}
```

22

DAO no Floor

```
import 'package:floor/floor.dart';

@dao
abstract class TaskDao {
  @Query('SELECT * FROM TaskEntity')
  Future<List<TaskEntity>>
  findAllTasks();

  @Query('SELECT * FROM TaskEntity
  WHERE id = :id')
  Future<TaskEntity?> findTaskById(int
  id);

  (...)

  (...)
  @insert
  Future<void> insertTask(TaskEntity
  task);

  @update
  Future<void> updateTask(TaskEntity
  task);

  @delete
  Future<void> deleteTask(TaskEntity
  task);
}
```

Database no Floor

```
import 'dart:async';
import 'package:floor/floor.dart';
import 'package:sqflite/sqflite.dart' as sqflite;

part 'app_database.g.dart';

@Database(version: 1, entities: [TaskEntity])
abstract class AppDatabase extends FloorDatabase {
  TaskDao get taskDao;
}
```

Gerando código

Floor usa geração de código.

```
dart run build_runner build
```

O arquivo gerado será:

```
app_database.g.dart
```

Inicializando o banco

```
final database = await $FloorAppDatabase  
    .databaseBuilder('app_database.db')  
    .build();
```

```
final taskDao = database.taskDao;
```

INSERT

```
final task = TaskEntity(  
    title: 'Estudar armazenamento local',  
    done: false,  
);  
  
await taskDao.insertTask(task);
```

UPDATE

```
final updatedTask = TaskEntity(  
    id: 1,  
    title: 'Estudar Floor',  
    done: true,  
);  
  
await taskDao.updateTask(updatedTask);
```

DELETE

```
final task = TaskEntity(  
    id: 1,  
    title: 'Estudar Floor',  
    done: true,  
);  
  
await taskDao.deleteTask(task);
```

29

Floor

Vantagens

- Organização clara
- Usa SQL
- Bom para dados estruturados
- Facilita CRUD
- Permite queries específicas
- Ajuda a separar acesso a dados da interface

Desvantagens

- Exige geração de código
- Exige conhecimento básico de SQL
- Mudanças de estrutura exigem migração
- Mais verboso do que Hive
- Não é ideal para dados muito flexíveis

30

Hive

- Hive é um banco NoSQL local chave-valor.
- Ele não usa tabelas SQL.
- Ele usa boxes.

Box: settings

```
darkMode → true  
language → "pt_BR"
```

Box: tasks

```
1 → Task(...)  
2 → Task(...)
```

Instalando Hive

- No terminal:

```
flutter pub add hive hive_flutter  
flutter pub add dev:hive_generator dev:build_runner
```

- Importações:

```
import 'package:hive/hive.dart';  
import 'package:hive_flutter/hive_flutter.dart';
```

Inicializando Hive

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Hive.initFlutter();  
  
  runApp(const MyApp());  
}
```

33

Salvando objetos no Hive

```
import 'package:hive/hive.dart';  
  
part 'task_model.g.dart';  
  
@HiveType(typeId: 0)  
class TaskModel extends HiveObject {  
  @HiveField(0)  
  String title;  
  
  @HiveField(1)  
  bool done;  
  
  TaskModel({  
    required this.title,  
    required this.done,  
  });  
}
```

34

Registrando adapter

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Hive.initFlutter();  
  
  Hive.registerAdapter(TaskModelAdapter());  
  
  await Hive.openBox<TaskModel>('tasks');  
  
  runApp(const MyApp());  
}
```

35

Gerar adapter:

```
dart run build_runner build
```

36

CRUD com Hive e objetos

```
final box = Hive.box<TaskModel>('tasks');

await box.add(
  TaskModel(
    title: 'Estudar Hive',
    done: false,
  ),
);

final tasks = box.values.toList();

final firstTask = box.getAt(0);
firstTask?.done = true;
await firstTask?.save();

await firstTask?.delete();
```

37

Hive

Vantagens

- Simples
- Rápido para chave-valor
- Bom para cache
- Bom para dados offline
- Não precisa escrever SQL
- Funciona bem com objetos Dart

Desvantagens

- Não é relacional
- Consultas complexas são limitadas
- Relações entre entidades ficam manuais
- Mudanças em adapters exigem cuidado
- Não substitui bem SQL em cenários relacionais

38

Firestore

- Firestore é uma plataforma de backend para apps.
 - Firestore Core
 - Firestore Authentication
 - Cloud Firestore
- Ideia principal:
 - O Flutter fica com a interface
 - O Firestore fornece autenticação e banco remoto.

39

Firestore

- Firestore é um banco NoSQL em nuvem.
- Ele organiza dados em:
 - Collections
 - Documents
 - Fields
 - Subcollections

```
users
  userId123
    name: "Ana"
    email: "ana@email.com"

tasks
  taskId456
    title: "Estudar
Firestore"
    done: false
    userId: "userId123"
```

40

Modelo mental do Firestore

- Firestore não usa tabelas.
- Ele usa coleções e documentos.

Tabela users

	id	nome	email	idade
Linha 1	1	Ana	ana@email.com	25
Linha 2	2	Bruno	bruno@email.com	30



41

Instalando Firebase CLI e FlutterFire CLI

Instalando Firebase CLI e FlutterFire CLI

```
firebase login
```

```
dart pub global activate flutterfire_cli
```

```
flutterfire configure
```

O comando flutterfire configure gera o arquivo:

```
lib/firebase_options.dart
```

42

Instalando pacotes Firebase

```
flutter pub add firebase_core  
flutter pub add firebase_auth  
flutter pub add cloud_firestore
```

Inicializando Firebase no Flutter

```
import 'package:firebase_core/firebase_core.dart';  
import 'firebase_options.dart';  
  
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  
  runApp(const MyApp());  
}
```

Salvando dados no Firestore

```
import 'package:cloud_firestore/cloud_firestore.dart';

class TaskRemoteDataSource {
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  Future<void> addTask({
    required String userId,
    required String title,
  }) {
    return _db.collection('tasks').add({
      'userId': userId,
      'title': title,
      'done': false,
      'createdAt': FieldValue.serverTimestamp(),
    });
  }
}
```

Mostrando dados na tela

```
import 'package:cloud_firestore/cloud_firestore.dart';

class TaskRemoteDataSource {
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  Future<void> addTask({
    required String userId,
    required String title,
  }) {
    return _db.collection('tasks').add({
      'userId': userId,
      'title': title,
      'done': false,
      'createdAt': FieldValue.serverTimestamp(),
    });
  }
}
```